

RPU – A Reasoning Processing Unit

Matthew Joseph Adiletta, Gu-Yeon Wei and David Brooks
Harvard University

Abstract—Large language model (LLM) inference performance is increasingly bottlenecked by the memory wall. While GPUs continue to scale raw compute throughput, they struggle to deliver scalable performance for memory bandwidth bound workloads. This challenge is amplified by emerging reasoning LLM applications, where long output sequences, low arithmetic intensity, and tight latency constraints demand significantly higher memory bandwidth. As a result, system utilization drops and energy per inference rises, highlighting the need for an optimized system architecture for scalable memory bandwidth.

To address these challenges we present the Reasoning Processing Unit (RPU), a chiplet-based architecture designed to address the challenges of the modern memory wall. RPU introduces: (1) A Capacity-Optimized High-Bandwidth Memory (HBM-CO) that trades capacity for lower energy and cost; (2) a scalable chiplet architecture featuring a bandwidth-first power and area provisioning design; and (3) a decoupled microarchitecture that separates memory, compute, and communication pipelines to sustain high bandwidth utilization. Simulation results show that RPU performs up to 45.3× lower latency and 18.6× higher throughput over an H100 system at ISO-TDP on Llama3-405B.

I. INTRODUCTION

Low-latency inference is critical for reasoning LLMs, which may generate thousands of tokens before reaching a final answer [8], [12], [66], [67]. Without fast token generation, reasoning tasks become slow, limiting the adoption of this powerful inference approach. Therefore, we introduce the Reasoning Processing Unit (RPU), a new chiplet-based system architecture that delivers orders of magnitude higher memory bandwidth than today’s compute-centric designs, addressing the core latency bottleneck in reasoning LLMs.

The first step toward low-latency LLM inference is to separate prefill and decode onto different systems, as in Dynamo [17] and the Splitwise execution model [50]. Prefill is compute-bound and highly parallel, making it efficient to run on today’s GPU architectures. Decode, in contrast, is inherently sequential and latency-sensitive. When prioritizing latency, decode systems must operate at low batch sizes, often as low as one. This low-batch regime is not a choice but a necessity, for two key reasons.

First, each query must compute attention sequentially during decode, so larger batch sizes lead to higher latencies. Long sequences are common in reasoning models, which makes this effect even more pronounced. Second, latency-optimized techniques like speculative decoding [37] are only effective at low batch sizes [20]. As a result, small batch sizes are essential for low latency. However, low-batch inference exacerbates the memory wall problem. For example, Figure 1 shows low-batch decode operates far below the H100’s compute roofline.

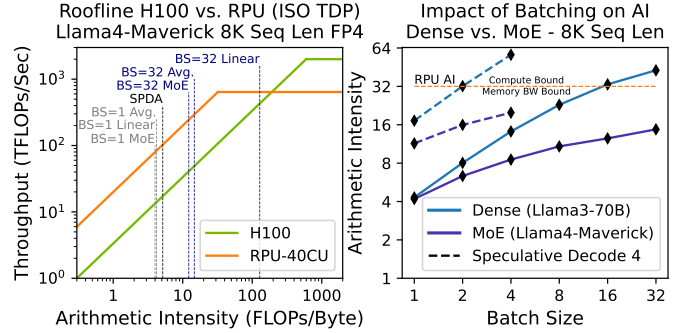


Fig. 1. RPU provides higher memory bandwidth than H100, which is required for low-latency decoding. Even up to BS=32, arithmetic intensity remains low, but requires the RPU to execute kernels which straddle the roofline.

System architects have responded to the decode bottleneck by cramming more HBM stacks into each package as the simplest way to chase bandwidth [14], [27], [61], [63]. However, HBM was not designed for inference. Its popularity was driven primarily by GPU training, pushing a roadmap focused on high bandwidth coupled with high capacity to store massive training datasets and large model checkpoints to feed dense compute architectures. Now, as applications shift toward inference, HBM’s high capacity per module drives up energy and cost, undermining scalability for low-latency inference.

Challenge 1 – Energy and Cost of Memory. In streaming workloads, over 74% of memory device energy is spent moving data across long internal wires and TSVs within the DRAM stack [43], [45]. Higher capacity HBMs increase internal paths, driving up energy per bit. This creates a fundamental mismatch for low-reuse streaming workloads, where minimizing energy per bit is critical for efficiency. Similarly, the cost of next-generation AI accelerators is increasingly dominated by HBM memory [48], [49]. Cost scales with capacity, as larger stacks require more silicon area and complex packaging.

The capacity required per socket depends on how a model is deployed (e.g., a single node, across a rack [6], or at datacenter scale [26]). If a model fits in one GPU, doubling the number of sockets doubles bandwidth, but it also halves the capacity needed per socket. We call this the *memory overprovisioning paradox*: a system-level design inefficiency where scaling out for bandwidth to achieve lower latency overprovisions capacity and drives up system cost and energy. The correct architectural approach is to provision memory based on the scale of the intended deployment. But with HBM as the only high-bandwidth memory available [29], architects are locked into a costly compromise – *buying bandwidth with capacity*.

Contribution 1 – HBM-CO: An HBM-style memory design which can be optimized to the bandwidth-capacity needs of low-batch LLMs, addressing energy and cost challenges of the memory wall (Section III). HBM-CO retains HBM’s internal bandwidth architecture and shoreline bandwidth, but selectively reduces capacity-driving structures such as ranks, banks and subarrays to optimize for the capacity needs of low-latency inference. These changes require minimal modification to the HBM stack, making HBM-CO a practical and manufacturable design point. This yields higher bandwidth per dollar and up to 2.4× energy efficiency than conventional HBM, despite a higher cost per GB. We quantify these tradeoffs using an analytical modeling approach based on [45], which estimates energy per bit and cost per module from wire-length scaling trends from HBM core-die floorplans [35], [47], [54].

Modularizing memory with HBM-CO unlocks a new design regime. Instead of provisioning large, monolithic capacity per socket, systems can scale bandwidth and capacity across many smaller stacks. But this flexibility introduces a new architectural challenge: how do we build a compute fabric that can glue together these HBM-CO modules, deliver the desired bandwidth, and stay within tight power and packaging limits? The answer requires rethinking how power and area are provisioned in modern accelerators.

Challenge 2 – Power and Area Provisioning for A Scalable Compute Fabric: Today’s HBM-based accelerators (e.g., H100 [14], MI300x [61]) allocate only 30-40% of system thermal design power (TDP) to memory interfaces, meaning that memory-bound workloads leave a large fraction of available power underutilized. These systems are also reticle-limited, which favor dense compute, but this leads to overprovisioned arithmetic and cache resources during bandwidth bound workloads. Additionally, memory bandwidth scales with die perimeter, not area, because each HBM stack requires a dense ring of high-speed IOs along the chip edge (shoreline) [10]. Reticle-limited designs minimize this perimeter-to-area ratio, which directly conflicts with the need for scalable bandwidth.

Alternatively, accelerators like Cerebras [64] and Groq [2] avoid shoreline constraints by using SRAM as main memory. However, SRAM’s low density drives up power, cost, and infrastructure overheads. For example, Cerebras requires four WSEs to host a 70B model, and Groq needs hundreds of processors. In contrast, DRAM provides the density to support large models at far lower system footprint.

Contribution 2 – RPU: A modular chiplet-based architecture that dedicates more power to memory interfaces and optimizes the compute-to-bandwidth ratio for low-latency token generation (Section IV). The RPU embraces emerging trends in package-level integration to achieve scalable memory bandwidth. Rather than concentrating compute in a monolithic die, the RPU distributes compute across many smaller chiplets. As a result, for the same compute die area, the RPU exposes nearly 10× more memory IO shoreline than the H100 (600mm vs. 60mm), enabling tighter coupling between memory and logic. Four compute chiplets are co-packaged with eight HBM-CO stacks, following package-level integration strategies pi-

oneered by modern GPU architectures [14], [22], [26], [52], [61]. Packages are composed at the board level, increasing bandwidth until blade-level power envelopes.

In parallel, the RPU reprovisions power and area. The RPU dedicates 70-80% of power to memory interfaces. This keeps the memory bandwidth bound power near the peak power and enables over 2× bandwidth at ISO TDP versus compute-centric GPUs. The RPU also aligns the compute-to-bandwidth ratio. By removing underutilized compute and cache resources, it improves area efficiency and reduces die cost. As shown in Figure 1, these RPU design choices shift the roofline down and to the left, which is a better fit for low-latency inference.

The RPU system-architecture make more bandwidth available; however, roofline availability is not the same as utilization. Realizing the full potential of the RPU system requires rethinking how we sustain bandwidth at scale.

Challenge 3 – Utilizing the Newly Available Bandwidth: Today’s systems struggle to use memory bandwidth effectively, particularly in low-batch LLM token generation [33], [68]. This is because small, distributed weight matrices limit streaming bandwidth [38], [52]. For example, the fused *gate/up* projection MLP layer in Llama4-Maverick contains just 168 million parameters ($5k \times 32k$). When *column-sharded* [65] across devices with TB/s of bandwidth, the roofline model predicts runtimes in the tens of microseconds. In practice, kernel launch and tensor-parallel communication latencies are often of similar magnitude, which limit bandwidth utilization [38].

Contribution 3 – Reasoning Core: A decoupled pipeline microarchitecture and custom ISA that fully saturates available memory bandwidth (Section V). Achieving high bandwidth utilization requires careful orchestration of data movement. The RPU Reasoning Core microarchitecture addresses this by separating dataflow for memory, compute, and network into independent pipelines, connected by on-chip buffers and coordinated through programmable pipeline arbiters. This decoupling allows each pipeline to make forward progress based on data readiness, without stalling on global barriers. At batch size 1 ($BS=1$), the RPU saturates memory bandwidth and achieves roofline performance. On the other hand, batch size 32 ($BS=32$), contains kernels that straddle the roofline, as shown in Figure 1, alternating between memory-bound SDPA and MoE layers and compute-bound Linear layers. Decoupled pipelines enable the RPU to absorb this phase imbalance into the buffer hierarchy and sustain throughput at the workloads average arithmetic intensity (AI).

Contribution 4: An end-to-end simulation framework for LLM inference on RPU (Sections VI- VIII). We developed a simulation framework, combining RTL-modeled compute kernels, an analytical energy model of HBM-CO memory, and an event-driven simulator [7], [55] that executes compiled LLM workloads via a custom RPU ISA. This framework captures transient dataflows, pipeline utilization, and synchronization stalls, which enables detailed architectural comparisons against modern GPUs. Compared to an H100 at ISO-TDP, RPU achieves up to 45.3× lower latency and 18.6× higher throughput on Llama3-405B at similar system cost.

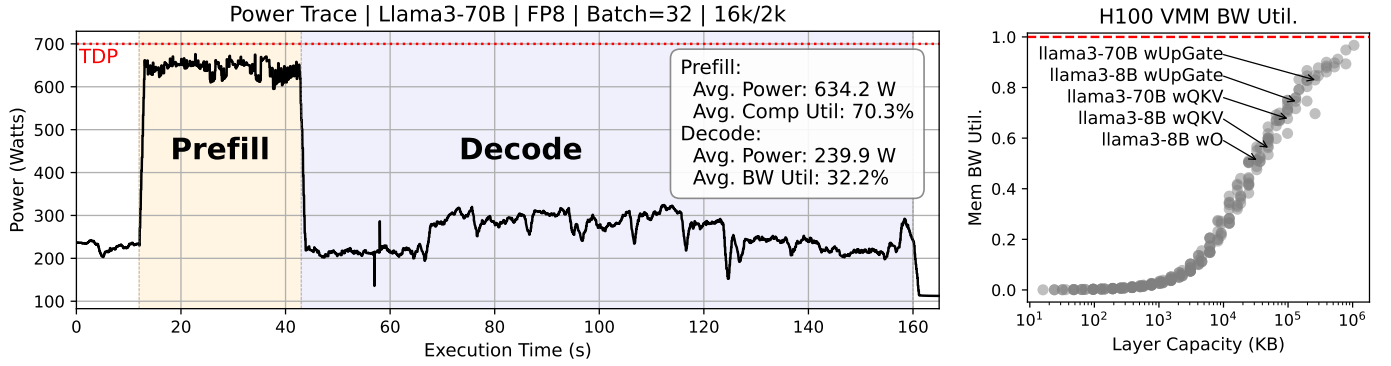


Fig. 2. Power and utilization characterization of H100 using NVML. Left: Power trace during distributed inference (4xH100) of Llama3-70B (Batch=32). Right: Isolated kernel profiling for memory bandwidth utilization across batch sizes and matrix dimensions (BF16).

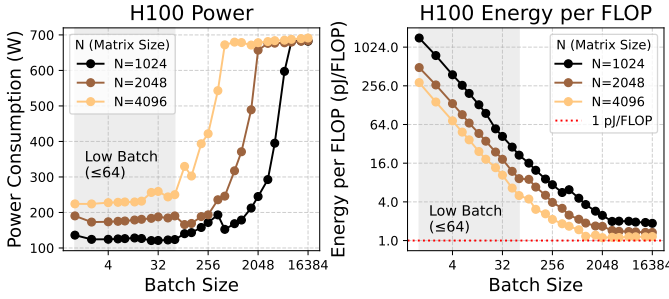


Fig. 3. Isolated kernel profiling for power consumption and energy efficiency across batch sizes and matrix dimensions (BF16).

II. MOTIVATION FOR A DECODE-OPTIMIZED SYSTEM

To illustrate the challenges of the modern memory wall on today’s GPUs, we profile the H100 using: (1) low batch LLM inference for end-to-end behavior, and (2) standalone dense-linear kernels to isolate bottlenecks.

Experimental Setup: We use NVIDIA’s NVML [3] to measure power on an H100 GPU running CUDA software stacks optimized for HBM3e. For full-model profiling, we profile Llama-70B with FP8 weights [5] on 4xH100s using vLLM [34] and NVIDIA Dynamo [17], running batch-32 inference with 16k prefill and 2k decode tokens (Figure 2, left). To isolate bottlenecks, we also benchmark representative dense-linear kernels compiled with PyTorch 2.2 [1].

Low Power Efficiency at Low Batch Sizes: Figure 2 (left) shows that during LLM prefill, the H100 uses 90% of its TDP and achieves high compute utilization. In contrast, the decode-phase only uses 34% of its TDP. This observation is reinforced by isolated dense-linear profiling in Figure 3 (left); batch sizes ≤ 64 consistently yield $<30\%$ TDP. The inability to fully utilize power suggests a critical mismatch between the H100 design and low-latency inference.

Low Energy Efficiency at Low Batch Sizes: Figure 3 (right) shows that while high AI, compute-bound kernels are energy efficient (~ 1.0 pJ/BF16 FLOP), this degrades by 10-1000 $\times$ for low-batch inference due to non-amortized data movement costs. HBM3e accesses alone account for 30-50%

of total energy [43], with additional losses from on-chip data movement across the H100’s large monolithic die [14]. The UMA memory system and randomized address mapping further inflate energy costs by physically increasing the distance data travels from memory to compute.

Inference Does Not Achieve Peak Memory-BW Utilization:

Our profiling shows that the H100 only utilizes 32% of its peak memory bandwidth during distributed LLM decode. This observation is consistent with prior work on low-batch inference [33], [52], [68] as well as NVIDIA self-reported benchmarks (20k/2k at 911 OTPS) [3]. Isolated experiments in Figure 2 (right) show that full bandwidth is only achieved when the working set exceeds ~ 1 GB, which is far larger than typical LLM matrices. Model sharding and reduced precision (e.g., 16-bit to 4-bit) further reduces each matrix’s footprint per GPU, leading to even lower bandwidth utilization.

Multiple factors contribute to low memory bandwidth utilization: kernel launch overheads become non-negligible for small kernel sizes [38]; long memory access latencies cannot be hidden behind compute; and inefficient vector broadcasts between SMs using shared memory limit throughput between layers. Together, these limitations show that the H100 memory system is not optimized for the decode phase of LLMs, underscoring the need for a decode optimized architecture.

III. CAPACITY OPTIMIZED HIGH BANDWIDTH MEMORY

Low-batch token generation latency is fundamentally limited by memory bandwidth. For dense models like Llama3, consider the case where the model’s memory footprint (weights and KV\$) fits perfectly within the systems memory capacity (100% capacity utilization). In this configuration, all memory capacity is actively used, and token generation latency is determined solely by how quickly that memory can be read. This scenario exposes a fundamental constraint: when memory is fully utilized, the minimum achievable latency is set by the ratio of bandwidth to capacity (BW/Cap). As a result, BW/Cap emerges as a key metric for evaluating and designing memory systems for bandwidth-bound inference. Higher BW/Cap enables faster access to the entire model, reducing latency and improving memory efficiency.

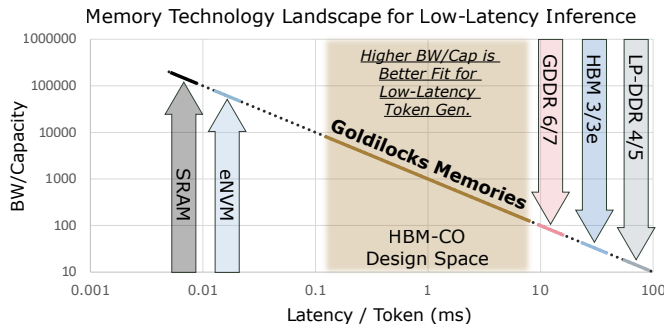


Fig. 4. Memory technology landscape comparing bandwidth per capacity versus latency per token with 100% capacity utilization for dense LLMs. A technology gap exists in the *Goldilocks* range for low-latency inference.

Modern memory systems fall short of the bandwidth-to-capacity ratios desirable for efficient low-latency inference. For example, achieving a 1ms token latency while fully utilizing memory would require a BW/Cap of approximately 1000, which is equivalent to 1TB/s per GB. In contrast, high-end memory technologies like HBM3e offer much lower BW/Cap ratios. For instance, a single HBM3e stack provides 1280GB/s of bandwidth and 48GB of capacity, yielding a BW/Cap of 27 [35]. To meet bandwidth targets, system designers must aggregate multiple stacks, which increases total memory capacity far beyond what the model requires and results in severe capacity underutilization. The fraction of memory actually used is proportional to the ratio between available and required BW/Cap. In this example, with a target of 1000 and available BW/Cap of 27, only 2.7% of capacity is effectively utilized.

This mismatch between the desired bandwidth and practical memory capacity defines the memory overprovisioning paradox: High-capacity DRAM-based memories like HBM, GDDR, and LP-DDR *buy bandwidth via capacity* – scaling and distributing weights across multiple memory modules to increase memory bandwidth, resulting in under-utilized capacity. Conversely, SRAM-based architectures *buy capacity via bandwidth* – offering extreme bandwidth, but struggling to fully utilize it due to excessive sharding across devices caused by limited storage density.

Figure 4 illustrates this design gap: no commercial memory technology occupies the high BW/Cap regime desirable for low-latency LLM inference.

Overprovisioned capacity also introduces energy and cost inefficiencies. Prior work [45] shows that 74% of HBM energy in streaming workloads is spent on internal data movement, with only 14% and 12% attributed to I/O and row activation. As capacity increases, internal wire lengths grow, raising energy per bit and reducing efficiency. In addition, memory cost scales with capacity due to more silicon area.

To address these challenges, memory capacity per device should become a tunable architectural parameter. LLMs differ widely in model size, sparsity, deployment context, and system constraints. Each use case has a different optimal BW/Cap profile, often beyond what current technologies can deliver.

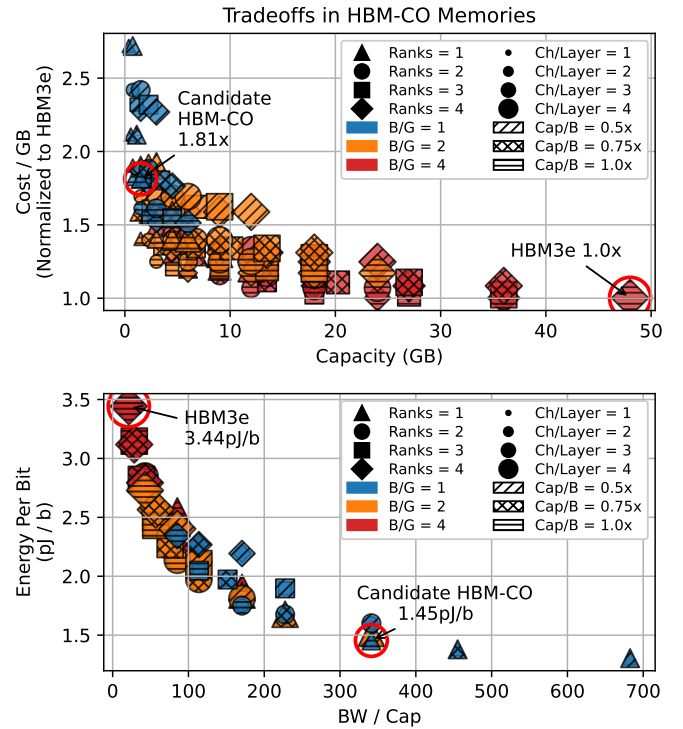


Fig. 5. Tradeoffs in HBM-CO memories, illustrating that high-BW/Cap memories are up to $\sim 2.5\times$ more energy efficient than an HBM3e device, but $\sim 1.8\times$ the higher cost per GB.

Decoupling capacity from bandwidth would allow system designers to provision memory precisely for application needs, improving performance, efficiency, and cost-effectiveness.

The Design Space of HBM-CO Memories: To fill the memory technology gap for low-latency inference, we propose a new class of memory devices: *Capacity-Optimized High-Bandwidth Memories* (HBM-CO). We analyzed the HBM architecture and identified parameters that impact a stacked memory’s bandwidth-to-capacity ratio [11], [29], [30].

HBM Ranks and Layers: Each rank consists of four stacked DRAM layers (dies). All layers in a rank contribute to higher memory bandwidth, each with separate channels. However, increasing the number of ranks adds memory capacity but does not increase bandwidth, since the interface is shared.

HBM Channels and Pseudo-Channels: A DRAM layer is partitioned into four channels, each of which is further split into two pseudo-channels (pCHs) for a total of eight pCHs per layer. The 8 pCH across 4 layers per rank fully saturate the memory bandwidth broken into a 32-pCh x 32b IO interface.

HBM Bank Groups, Banks, and Sub-Arrays: A pseudo-channel contains four bank groups, each with four banks. To sustain the full 32 GB/s bandwidth per pCH, only one active bank per bank group is needed using innovations such as sub-array level parallelism [31]. Four active bank groups per pCH are pipelined to delivers 256 bits per 1 GHz. Banks are composed of subarrays, which contribute to total capacity but do not impact bandwidth.

Key Insight to Change the BW/Cap of HBM: HBMs achieve

peak bandwidth per shoreline with just one active bank per bank group per pseudo-channel. This means capacity structures such as sub-arrays per bank, banks per bank group, and ranks can be parameterized without changing bandwidth.

Modeling Energy and Cost for HBM-CO: We developed an analytical HBM-CO model to capture tradeoffs in bandwidth, capacity, energy, and cost. Energy per bit was broken into four components: (1) Row Activation: 0.18pJ/bit for streaming workloads [11], [45]. We conservatively model HBM-CO with HBM3 timing and activation energy, leaving potential bandwidth and energy gains from its smaller core-die and sub-arrays for a future physical design study. (2) Data Movement: 0.2pJ/bit/mm, estimated from intra-die routing distances derived from HBM core-die floorplans [35], [47], [54]. (3) TSV Traversal: 0.148pJ/bit/layer, based on 0.8pF TSV capacitance and switching energy [28]. (4) I/O Interface: 0.25pJ/bit, drawn from UCIe specs and HBM3e datasheets [16], [43]. Cost is normalized against an HBM3e baseline [49], [68], scaling against silicon area and accounting for non-amortized costs such as base-die logic and TSV footprint. At lower capacities, these fixed costs dominate, impacting cost per GB more significantly. We validate our HBM-CO model against HBM3e [43] reported 3.44pJ/bit.

Design Space Takeaways: Figure 5 visualizes the tradeoffs in energy, cost, and BW/Cap for HBM-CO memories. A candidate Pareto-optimal HBM-CO memory has 768MB capacity, 256GB/s bandwidth (BW/Cap = 341), and 1.45pJ/bit energy. This device offers 2.4 $\times$ lower energy per bit than HBM3e while maintaining the same bandwidth per shoreline (GB/s/mm). This candidate memory BW/Cap leads to an ideal token latency of 2.9ms per token, falling in the middle of the *Goldilocks* memory range of Figure 4. An HBM3e system with the same performance would only utilize 7.9% of its capacity for inference with a dense LLM.

This efficiency comes at a cost. The candidate is 1.81 $\times$ more expensive per GB, seemingly violating the foundational DRAM principle of minimizing cost per bit. However, for low-latency inference, bandwidth per dollar is the important design metric. By trading 192 $\times$ capacity and 1.81 $\times$ higher price per GB, the resulting module is 35 $\times$ lower cost overall, achieving 5 $\times$ higher bandwidth per dollar than HBM3e.

IV. COMPUTE FABRIC FOR LOW-LATENCY INFERENCE

Distributed Vector-Matrix Multiplication (VMM) is the core operation in LLM token generation. Given an input vector $V \in \mathbb{R}^{1 \times K}$ and a weight matrix $W \in \mathbb{R}^{K \times N}$: $O = V * W$, $O \in \mathbb{R}^{1 \times N}$. For low-latency inference, this computation must be parallelized efficiently across devices to maintain fast, per-token response times. Prior work [65] has exploited the layered structure of AI models, where each layer’s output serves as part of the input to the next. Consider a system comprising C number of cores. Sharding W along its columns ensures that each core computes a disjoint portion of the output vector. The weight matrix is partitioned such that each core stores $W_i \in \mathbb{R}^{K \times \frac{N}{C}}$ and computes its corresponding output fragment $O_i = V * W_i$, $O_i \in \mathbb{R}^{1 \times \frac{N}{C}}$.

Since each core holds a portion of the output vector O , which serves as the input to the next layer (O_i becomes V_i), it can immediately begin computing on its local fragment for the next layer while simultaneously broadcasting its portion of V to other cores. This allows each core to progress with available data while receiving the remaining parts of V . This strategy mirrors Cannon’s algorithm for distributed matrix multiplication, where data movement and computation are interleaved to maximize efficiency.

To further increase parallelism, rows of W (K -dimension) can be distributed across G cores in a processing groups. Using this approach, each core stores weight shard $W_{j,i} \in \mathbb{R}^{\frac{K}{G} \times \frac{N}{C}}$ to compute a partial output $O_{j,i}$, requiring a reduction step to sum the intermediate results $O_i = \sum_{j=1}^G O_{j,i}$. This reduction will always appear on the compute-network critical path, unlike the prior network-broadcast.

Figure 6 illustrates the proposed RPU chiplet-based architecture, designed to accelerate distributed VMM for low-latency inference. The RPU tightly integrates compute and memory across multiple hierarchy levels – cores, compute units, packages, and ring stations – to form a scalable and efficient system architecture.

Compute Unit and Reasoning Core: The Compute Unit (CU) is the fundamental building block of the RPU, providing tightly coupled compute and memory resources. Each CU is constructed with one compute chiplet and two HBM-CO chiplets, connected through advanced packaging such as EMIB [40] or CoWoS-L [25]. The module provides dual 256 GB/s memory shorelines, delivering consistent bandwidth per interface while offering customizable HBM-CO capacity.

The particular HBM-CO chiplet visualized in Figure 6 is derived from the HBM core-die shown in Figure 2 of [47], compacted by reducing banks per group from four to one, ranks from four to one, channels per layer from four to one, but keeping four layers per rank. Physically, the design reduces the DRAM array region and channel shoreline proportionally, while the TSV, command, and peripheral logic regions are unscaled, occupying roughly one-third of the total die area.

The compute-to-bandwidth ratio for a CU was determined empirically for low-latency inference using MXFP4 formats. We found that 32 OPs/Byte maximized utilization (Figure 1); higher ratios offered little benefit and only increased design complexity, silicon area, and energy cost. Thus, each 256 GB/s shoreline requires 8 TOPs of compute throughput.

The 256 GB/s shoreline can easily accommodate 512 MAC units along the same horizontal span while leaving adequate space for routing – defining the *compute shoreline*. To reach the target 8 TOPs, we stack 16 rows of MACs, organized into 8 reasoning cores. Each reasoning core comprises four 8x8 tile-multipliers (TMACs) and connects to its own HBM-CO memory pseudo-channel delivering 32 GB/s of memory bandwidth. This vertical stacking keeps routing paths short and avoids congestion along the bandwidth edge. Using both the top and bottom chip edges doubles the number of cores per CU while preserving a balanced area-to-perimeter ratio.

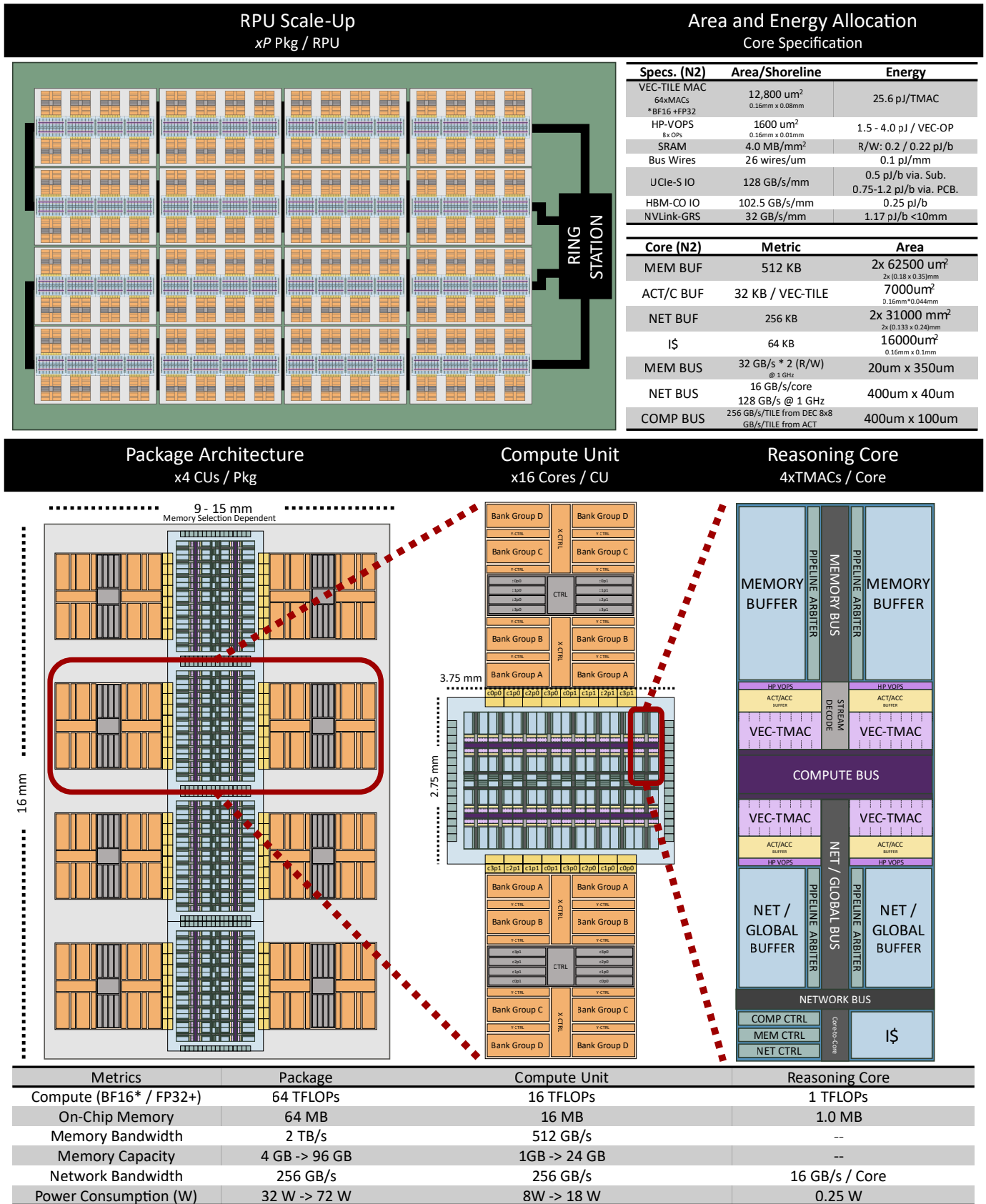


Fig. 6. Proposed RPU system architecture for low-latency LLM inference, featuring a chiplet-based design that rebalances compute, memory, and network resources. Each level of the hierarchy, from core micro-architecture to compute units, packages, and ring-station scale-up, is co-optimized for energy-efficient, cost-effective, and scalable memory bandwidth.

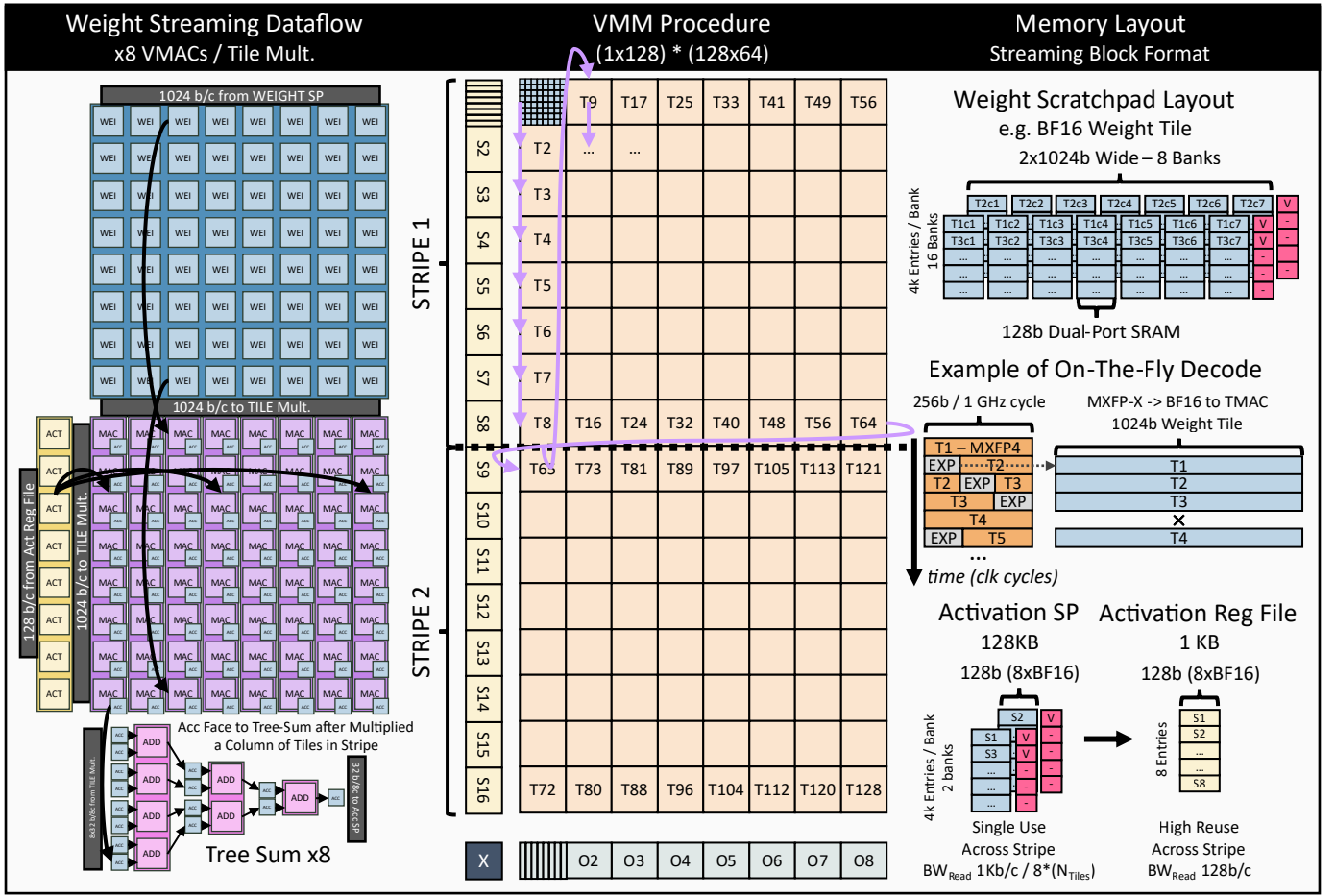


Fig. 7. Vector-Tile weight streaming dataflow and VMM procedure following a stripe-based execution. Arrows in *Weight Streaming Dataflow* indicate how activations and weights are moved into TMAC unit – activations are broadcast across columns while weights are element-wise moved. The arrows in the *VMM Procedure* indicates the order tiles are processed – column-wise first until eight rows of tiles are processed, then the next column starts, proceeding until all the columns in a stripe are completed.

Package Architecture: Four CUs are integrated onto a single package substrate, each equipped with its pair of dedicated HBM-CO memories offering 2 TB/s of memory bandwidth. At the package level, compute chiplets form a segment of the *outer ring* hierarchy. Vector fragments within a CU are forwarded to neighboring CUs in the same package through energy-efficient, short-reach UCle interconnects [16]. To minimize communication latency, each core includes a custom DMA engine optimized for fast inter-chiplet transfers, achieving latencies of ≤ 10 ns per CU-to-CU hop, which is similar to prior works [9], [23], [58], [71]. Each compute chiplet uses a unified UCle-S physical interface with segmented drivers: in-package links run at low voltage and high frequency (0.5 pJ/bit), while off-package links operate up to 16 GT/s with 0.75-1.2 pJ/bit energy [16], [51], defining the system’s outer-ring bandwidth at 128 GB/s/mm.

RPU Scale-Up: Multiple packages are soldered onto a PCB to form the outer ring topology, connected via a Ring Station. Communication between packages leverages PCB-routed interconnects, designed specifically for short-reach (< 10 mm) data transfers. A secondary purpose of the Ring-Station is to

network outside the system (e.g., 100Gb Ethernet).

An RPU is defined as a scalable compute system, composed of multiple co-packaged CUs, assembled on a board. Similar to how GPUs scale across datacenter and edge deployments by varying the number of CUDA cores, RPUs scale by composing different numbers of CUs. Our modular architecture enables flexible configurations to meet diverse performance, capacity, energy, and cost targets.

V. MICRO-ARCHITECTURE

NUMA Domains and Data Dependent Synchronization: A central design principle of our microarchitecture is a fully NUMA-based system. Each compute core within a CU forms an independent NUMA domain, without shared memory between cores. All data movement across domains is explicitly managed via software-programmable DMA engines and data-dependent synchronization. This eliminates coherence overhead, enables deterministic execution, and ensures scalable performance for dataflow-dominated workloads like LLM inference. Thus, the RPU favors *bespoke datapaths over generalized programming models*.

NUMA at All Scales: Each core includes three programmable data pipelines, each operating within its local NUMA boundary. The Memory DMA transfers data between the core’s dedicated HBM-CO memory channel and its memory buffer. The Compute DMA reads from memory or network buffers and feeds data into the compute pipeline. The Network DMAs manage all inter-core and inter-chiplet communication, linking each core to neighboring cores within a CU and the positionally aligned core in adjacent CUs. Incoming data is written to the network buffer and may be consumed locally and/or forwarded using custom forwarding instructions. This supports efficient collectives and data reuse across chiplets.

Pipeline Arbiters: We developed Pipeline Arbiters to synchronize decoupled memory, compute, and network pipelines. These lightweight, software-managed mechanisms are embedded within each core’s SRAM buffer. Each SRAM buffer entry includes a 2-bit valid counter that tracks the expected number of asynchronous consumers. DMA operations are programmed with a *valid_count* when writing and may optionally enable a *check_valid* flag to stall if the target address is occupied. On the read side, consumers can use *check_valid* to stall until data is ready and optionally decrement the valid counter after access. For example, a Network DMA may set *valid_count*=2 since activations will be consumed by (1) the compute pipeline and (2) asynchronously forwarded to neighboring cores.

To guarantee mutual exclusion, each buffer entry is accessed through a hardware-enforced arbitration mechanism that serializes requests from multiple consumers. Accesses are prioritized using a software-configurable policy, ensuring that only one DMA engine can read, write, or update the valid counter at a time. This enforces atomicity at the buffer-entry level and prevents race conditions across the memory, compute, and network pipelines. By managing synchronization through software-defined counters and flags, Pipeline Arbiters enable fine-grained, data-driven execution between NUMA domains with blocking and non-blocking semantics.

TMAC and HP-VOPs: The vector-tile MAC (TMAC) is the core computational unit for accelerating the VMM kernel, as shown in Figure 7. Each TMAC consists of 64 MAC units arranged in an 8×8 array, performing BF16 multiplies with FP32 accumulations. This structure allows one activation vector to be broadcast across 8 columns of the weight matrix, computing 64 MACs per cycle using a weight-streaming, output-stationary dataflow.

To maximize on-chip reuse of activation data and minimize accumulation write-back pressure, the VMM algorithm is organized into stripes. A stripe is a groups of 8 vertically stacked tiles spanning all columns of the weight shard. Activation shards per stripe contains 64 BF16 values, stored in a dedicated register file close to the tile multipliers. These values are initially fetched from the network buffer, then reused across all tile columns before being retired.

The tile multipliers first iterate over the tile-rows within a stripe. After processing a column of tiles, the accumulated face is reduced via a column-wise (3-stage) tree sum. These results are written back to a local register file to be read back

for the next stripe, leveraging the fact that each core typically operates on small output shards (<256 elements) in highly distributed VMMs. Once all the weight matrix columns of a stripe are computed, the next activation stripe shard is loaded from the network buffer, and the process repeats.

This striping approach is essential for three reasons: (1) Traversing columns first (inner-product style) would require the full activation vector to be stored on-chip, stalling compute during the vector broadcast across all CUs. (2) Traversing rows first (outer-product style) would result in high writeback bandwidth due to frequent partial sum updates. (3) By processing one stripe at a time, we minimize on-chip bandwidth requirements and enable fine-grained overlap of computation and communication; the next activation shard is collected in the network buffer while compute works on the current shard.

In addition to the tile multipliers, each core includes a general-purpose, high-precision (FP32) vector operations (HP-VOPs) accelerator, enabling support for key functions in LLM workloads (e.g., SiLU, GeLU, normalization, and rotary embeddings). Because overall performance is dominated by memory bandwidth, we can afford to allocate area to high-precision computation without significant impact on energy or latency. This enables numerical accuracy, particularly important for operations sensitive to precision loss such as attention.

Stream Decoder: To reduce latency and storage overhead, weight tiles are stored in compressed block formats in memory and transferred on-chip to the memory buffer by the memory DMA engine. Next, the compute DMA streams compressed weights into the *Stream Decoder*, which performs on-the-fly dequantization, converting block-quantized values into standard BF16. This continues until a full batch of 64 BF16 values is reconstructed, corresponding to a single weight tile. Once dequantized, the tile is broadcast across all active tile multipliers within the core via a 1024-bit wide compute bus.

Our stream decoder supports on-the-fly dequantization of multiple formats, including BFP [53], MxFP [15], and NxFP [39], with configurable bitwidths ranging from 4 to 8 bits. This flexibility allows us to efficiently compress weights off-chip while preserving the ability to compute at full precision on-chip, minimizing off-chip capacity and energy without compromising accuracy.

VI. RPU SOFTWARE STACK AND SIMULATION

RPU ISA and Compiler: The RPU software stack provides a deterministic compilation flow from PyTorch graphs to hardware execution. The RPU ISA hardens optimized vector-matrix and elementwise dataflows directly into hardware, exposing each as a single CISC-style instruction. Each instruction specifies operand addresses, tensor dimensions, and data types, while the hardware executes a fixed streaming schedule with near-roofline utilization. Computation follows a *push-based* dataflow: DMA engines deterministically inject data into buffers, and pipelines advance when inputs are ready. Hardware-managed pipeline arbiter flags are embedded within each instruction to synchronize compute, memory, and

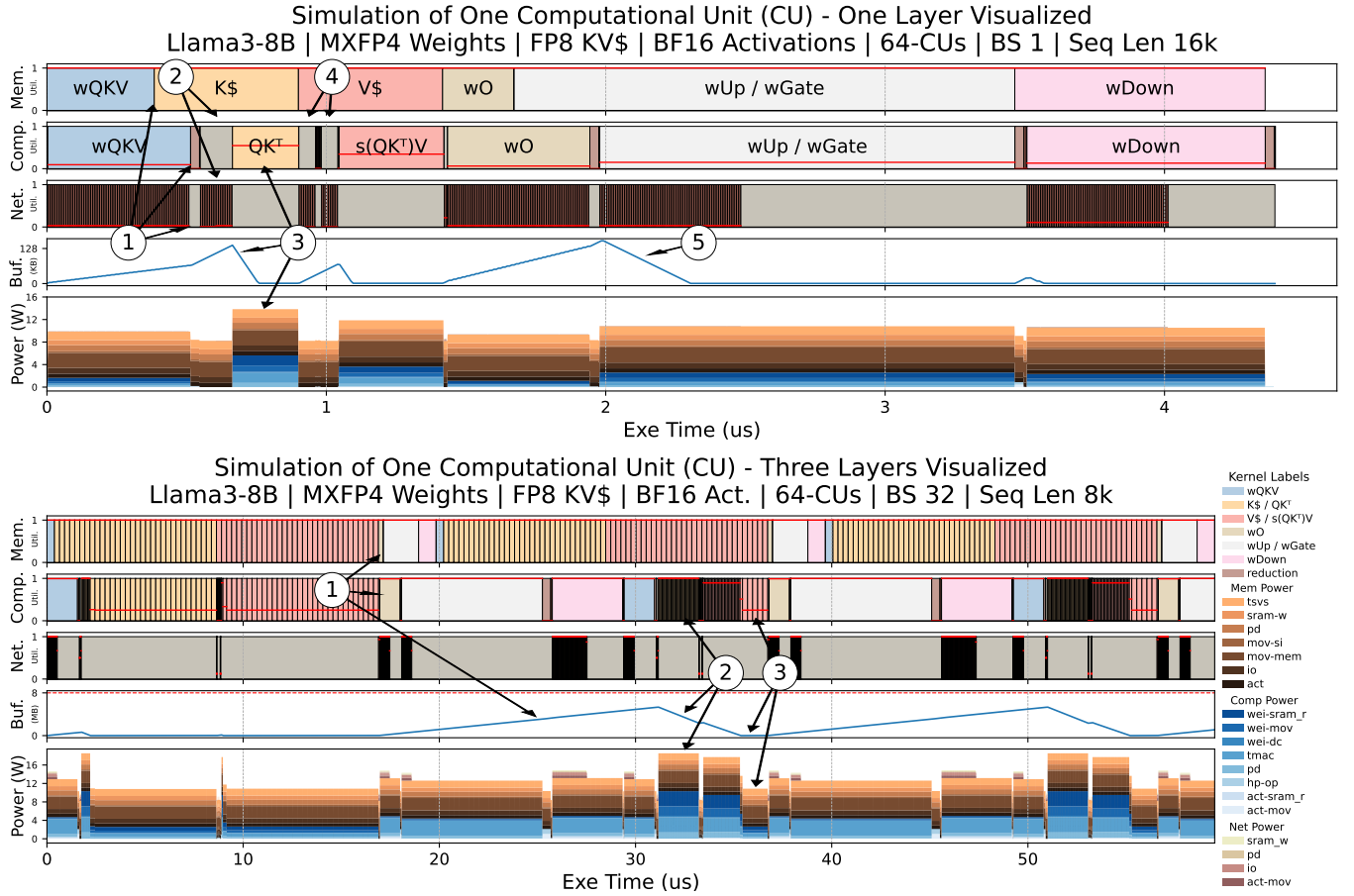


Fig. 8. Simulation of one CU in a 64-CU system running Llama3-8B. Top: Batch size 1, seq len 16k. Bottom: Batch size 32, seq len 8k. Each timeline shows memory, compute, and network utilization, buffer usage, and power. Memory power dominates total system power, highlighting the RPU's unique power provisioning. The plot also shows that batch size 32 generates tokens $\sim 13\times$ slower than batch size 1, primarily due to sequential KV\$ computations.

network pipelines, eliminating the need for software polling and ensuring deadlock-free progress.

A lightweight Python compiler traces PyTorch operations and lowers them to RPU primitives. For example, a `torch.nn.Linear` layer compiles into a three-stage microkernel – Loading, Looping, and Launching – that programs DMAs, drives the VMM pipeline, and forwards activations, respectively. The compiler statically orders all DMA and compute instructions, pre-shards and quantizes weights, and generates synchronized instruction streams for the memory, compute, and network pipelines. Together, the ISA and compiler form a compact, deterministic toolchain that provides predictable, near-roofline performance for token generation.

Deployment: Each RPU core includes a lightweight instruction-fetch pipeline that executes a small set of long-running instructions for a full LLM. This enables fully autonomous execution, eliminating the host-driven offload model used by GPUs. The host processor performs only coordination tasks such as transferring KV\$ from the prefill engine into RPU memory. After each transformer layer, the RPU triggers an interrupt to the host and reports generated tokens or completed queries and returns the corresponding KV caches.

RTL Simulations: We implement a proof-of-concept RPU in SystemC using Catapult HLS [62], [69], targeting TSMC N16 and projecting to N2 using published scaling factors [19], [59], [60]. RTL simulation includes a single-core CU, multi-CU packaging, and board-level integration. Key microkernels (e.g., VMM, DMA) are synthesized using VCS, Design Compiler, and PowerPro to extract calibrated energy and area. SRAM and interconnect energy are modeled analytically, and memory energy is provided by our HBM-CO analytical model, which captures the energy and bandwidth tradeoffs of capacity-optimized memory devices.

Compiled PyTorch transformer layers were executed on the RTL model to verify functional correctness and dataflow behavior. However, full-system cycle-accurate simulation remains computationally intensive: simulating a $2k \times 2k$ VMM on a 4-core RPU requires approximately 6.5 minutes, making end-to-end design space exploration (DSE) for LLM inference impractical at the RTL level.

Event Driven Simulation: To address RTL simulation latency, we developed a higher-level event-driven simulator that reproduces the behavior of the RTL model using symbolic transactions that capture address, size, and type instead of real

tensor data. The simulator models all key microarchitectural events – data transfers, stalls, and arbitration – using parameters calibrated to RTL throughput, bandwidth, and latency.

The event-driven simulator runs orders of magnitude faster than RTL while matching its latency and power estimates. It supports full-model DSE across models, batch sizes, sequence lengths, memory devices, and scales of deployments, while exposing transient behaviors such as buffer occupancy, pipeline stalls, and synchronization delays (Figure 8). The simulator also serves as a debugging and validation tool: violations of data dependencies appear as execution stalls, allowing developers to visually trace and correct synchronization issues in the simulation framework prior to deployment.

Simulation Results: Figure 8 shows a simulation of the first transformer layers in Llama3-8B on a 64-CU RPU using the candidate HBM-CO from Section III. We compare batch sizes 1 and 32, breaking down execution across memory, compute, and network pipelines, along with buffer and power traces. Each row represents a single CU, with red lines indicating average utilization per kernel. We highlight instances where the RPU’s decoupled pipelines enable out-of-order execution between memory, compute, and communication to unlock behaviors that conventional architectures cannot exploit.

Batch Size 1: ① During the first VMM ($wQKV$) execution is bounded by network-latency to broadcast activations across all CUs. This is reflected in the timeline by low network bandwidth utilization, and early memory-pipeline completion. Traditional architectures would stall the memory; however, the decoupled memory pipeline enables the RPU to simply continue fetching weights from memory to the on-chip buffer, while the compute pipeline lags the memory stream by $\sim 80\text{kB}$ waiting for activations to arrive from the network. During this period, power consumption is dominated by reading weights from memory: $\sim 6.7\text{W}$ at full BW / CU (512 GB/sec) and $\sim 1.7\text{pJ/b}$ datapath to write to the memory-buffer.

② Prior to the QK^T computation, compute stalls as each CU gathers its shard of the Q , K , and V vectors – 32 Q heads and 8 KV heads distributed across 64 CUs means each Q -vector spans two CUs while KV -vectors span eight CUs. Similarly, during ④, the compute stalls waiting first for a distributed *max* collective to calculate *softmax*, then an *exp-sum* reduction across the CUs sharing the 8 GQA-heads. These examples of cross-CU synchronization and network delays stall the compute pipeline, while the memory pipeline continues to prefetch and move weights and KV to the on-chip buffer. These types of network latency-bound periods are common in tensor-parallel distributed systems, where latency-bound network collectives are often on the orders of $\mu\text{seconds}$, leading to periods of fully stalled execution. The RPU allows the memory pipeline to continue prefetching ahead of the compute stream, effectively eliminating any incurred network collective overheads.

③ During the QK^T computation, each Q -vector (4 per GQA head) is assigned to a TMAC, while prefetched K entries are broadcast from the memory buffer. Similarly, in ⑤ the $wUp/wGate$ phase drives compute to full utilization

while processing the accumulated weights. This is a unique opportunistic moment for the RPU, as decoupled pipelines enabled the memory-bandwidth to stay fully saturated throughout network delays and compute stalls. Later, this enables the compute to play “catch-up” until all the available data is consumed from the on-chip buffer, returning to the memory-bandwidth bound performance. Power during this period rises due to higher compute utilization, from $\sim 1.5\text{W}$ to $\sim 5\text{W}$ per CU, reflecting full datapath activity. Once the buffer is drained, compute utilization returns to the memory-bound performance.

Batch Size 32: ① With a larger batch size, weight matrix operations become compute-bound. Specifically, during $wUp/wGate$, weight are read from memory in $\sim 2\mu\text{s}$, while compute is $\sim 4\times$ longer. While the RPU is compute-bound operating on the $wUp/wGate$ computation, the memory pipeline prefetches ahead, streaming in KV and filling each CU’s buffer with $\sim 6\text{MB}$ of weights totaling $\sim 384\text{MB}$ system-wide. This lookahead window is beyond the capabilities of GPU architectures like H100, which lack both the on-chip capacity and pipeline decoupling to absorb such deep prefetching.

② After a sequence of compute-bound weight matrix multiplications, the attention computation begins, entering a KV -intensive phase. Unlike weights, KV entries are query-unique, offering reuse only among GQA heads. Thus, this phase is inherently memory-bandwidth-bound. However, KV has already been prefetched on-chip which allows the system to stream KV directly from the on-chip buffer and operate at a compute-bound performance. As a result, the buffer drains rapidly, as the compute “catches-up” to the memory stream.

A batched LLM decode system naturally alternates between compute-bound weight layers and memory-bound KV layers, leading to pipeline underutilization on traditional architectures. Our design breaks this limitation by *smoothing out the bimodal workload* (compute-bound weights vs memory-bound KV) and *absorbing phase-imbalance into the memory buffer*, letting decoupled pipelines handle them independently. By smoothing performance across layers, we enable sustained utilization of both memory and compute. Without this buffering strategy, overall latency would increase by up to $1.6\times$.

③ Once the buffer is drained, compute returns to the memory-bound performance and power drop accordingly, marking the end of the amortization window. Larger batch sizes allow deeper prefetching, leading to a tradeoff in sequence length versus batch size to fully saturate bandwidth.

VII. SYSTEM IMPLICATIONS OF HBM-CO

HBM-CO memories improve efficiency at the device level, but their true benefits emerge only through full-system evaluation. Figure 9 shows energy per inference (y-axis) versus system memory capacity for a 64-CU RPU. HBM-CO memory configurations form a Pareto frontier showing the energy-capacity tradeoff; non-optimal points are omitted for clarity. Capacity reductions are progressively applied to an HBM3e-like memory to traverse the Pareto frontier. The best capacity reduction strategy is annotated between configurations. The

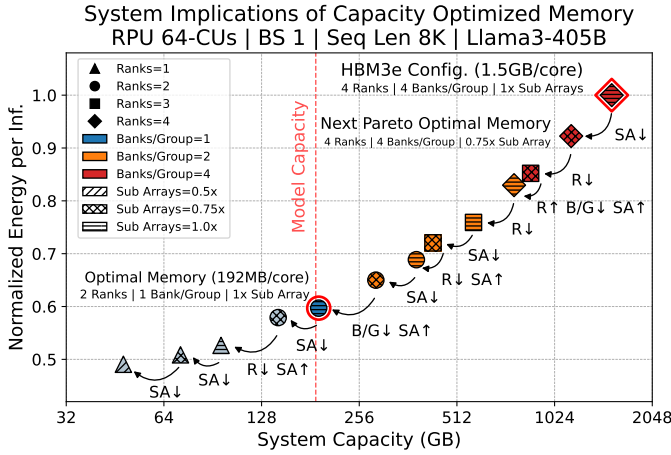


Fig. 9. Pareto frontier of HBM-CO memories for Llama3-405B inference on 64-CU RPU, annotated by stepwise changes in optimal HBM-CO. These represent the set of HBM-CO chiplets useful for a memory-chiplet ecosystem.

optimal HBM-CO has the smallest device capacity that meets the system-level requirement to store the target model.

For a 64-CU RPU running Llama3-405B with a single query and an 8k sequence length, the optimal HBM-CO configuration has a memory capacity of 192 MB per core. Compared to HBM3e, HBM-CO reduces the energy per bit by 2x from the memory cell to the IO, while at the system-level the energy per inference improves by 1.7x due to memory dominating the energy consumption. A similar tradeoff exists for system cost. Despite a 1.6x higher cost per GB, reduced capacity HBM-CO yields a 5.2x decrease in per-device cost, translating to a 4.3x total system cost reduction when factoring in compute, interposer, and substrate.

As illustrated in Figure 9, several HBM-CO configurations offer even lower energy per inference but remain inaccessible at the current 64-CU scale due to their limited memory capacity. Unlocking these more energy-efficient memories requires increasing the number of CUs, thereby decreasing the required memory per CU.

A key goal in deploying the RPU is to maintain workload flexibility without proliferating hardware SKUs. In the emerging chiplet ecosystem, the HBM-CO designs along the Pareto frontier of Figure 9 are sufficient to cover the useful BW/Cap design space. These chiplets can be mixed and matched at the package level to enable design customizations without fabricating a new ASIC. Figure 10 extends this idea by showing how to select among these variants for a given workload.

Figure 10 (top) is an HBM-CO SKU selection map for a 64-CU RPU running Llama4-Maverick. Each memory chiplet has a fixed bandwidth interface, resulting in a total system bandwidth of 32 TB/s. Given this fixed bandwidth, system capacity is optimized by selecting the most efficient HBM-CO chiplet configuration from Figure 9, to minimize both energy per inference and overall cost while satisfying capacity for each batch size and sequence length combination.

High BW/Cap SKUs maximize efficiency but limit the range

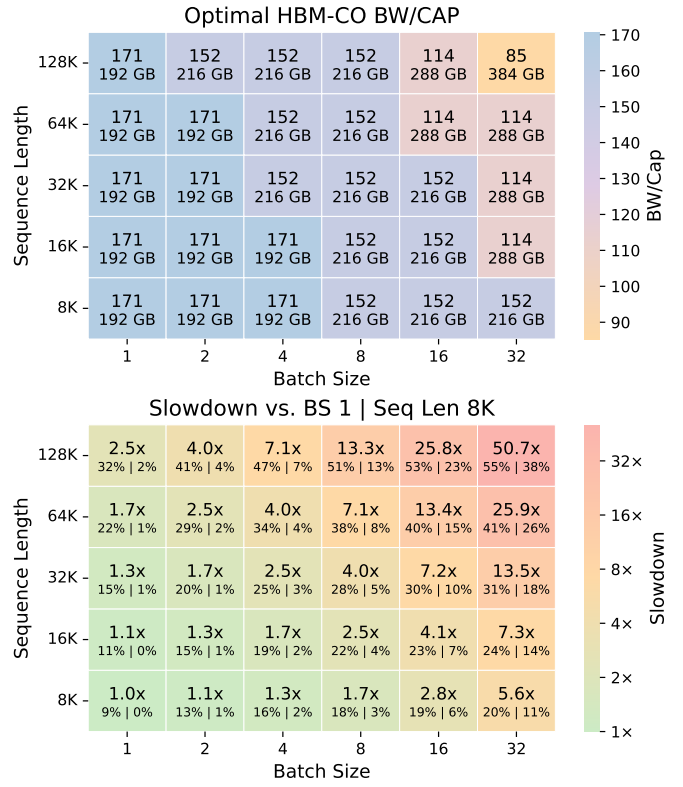


Fig. 10. RPU with 64 CUs running Llama4-Maverick showing batch size versus sequence length, comparing optimal HBM-CO BW/Cap and slowdown relative to BS=1, Seq Len=8K. Slowdown sub-metrics indicate the fraction of capacity used for KV cache versus active parameters and total capacity.

of supported batch and sequence lengths, while lower BW/Cap SKUs trade some efficiency for broader capacity coverage. Importantly, Figure 10 (top) shows that high-BW/Cap memories (5-6x HBM3e) are better suited for long-context, low-batch inference, which underscores the capacity overprovisioning of using off-the-shelf HBM3e. Increasing the number of CUs raises the optimal BW/Cap, enhancing efficiency at scale.

Figure 10 (bottom) quantifies how batching and sequence length impact latency. As batch size or sequence length increase, the per-query token generation latency increases. This is illustrated by tools such as InferenceMax [56], highlighting that low-batch inference is key for low latency. Longer sequences also intensify bandwidth pressure during attention – more than 50% of the active parameters are KV\$ for BS=8 128k. Therefore, the relative efficiency gap between the RPU and conventional GPUs widens due to the RPU’s bandwidth advantage, underscoring its use for long-context, low-latency inference.

VIII. STRONG SCALING ANALYSIS

Strong Scaling Analysis: We conduct a strong-scaling study by varying the number of CUs. Speedup is reported relative to the smallest configuration capable of fitting each model. Figure 11 shows results for Llama models, compared against an NVIDIA H100 at ISO TDP using the methodology

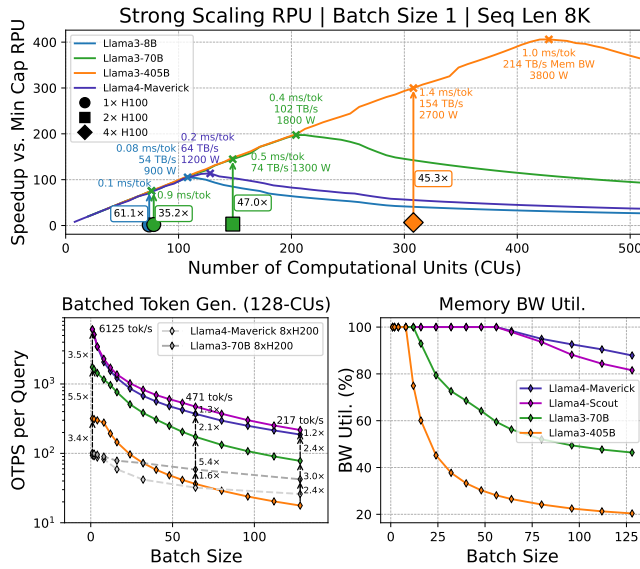


Fig. 11. Top: Strong scaling for Llama models under ISO TDP vs H100. Bottom: Output tokens per second per query (8xH200 from [4]) and bandwidth utilization versus batch size.

from Section II with 4-bit weights and 16-bit activations [18] and full tensor-parallelism.

Batch Size 1 – Fastest Thinking Speed: Batch size 1 represents the fastest possible “thinking speed” of a model. At ISO TDP, the RPU significantly outperforms H100 inference. Notably, the RPU latency is 47.0x faster than a 2xH100 at 1400W TDP for Llama3-70B and 45.3x faster than a 4xH100 at 2800W TDP for Llama3-405B. The 405B example is illustrated in Figure 11 (top) by the orange diamond (4xH100s) aligned to a 308 CU RPU system at ISO TDP.

Even more compelling is the peak performance of the RPU, achieved by scaling to the optimal number of CUs for each model: Llama3-70B at 204 CUs achieves 0.4 ms/token, and Llama3-405B at 428 CUs achieves 1.0 ms/token. Llama4-Maverick at 128 CUs achieves 0.2 ms/token. These are the fastest token generation latencies reported to date for these models. *Notably, we are the first system capable of sustaining over 200 TB/s of tensor-parallel memory bandwidth during inference for a 405B parameter model.*

Beyond these scales, performance plateaus as broadcasting the activation becomes the bottleneck. To overcome this limit, we propose two future directions: 1) Reduce on-chip forwarding latency. 2) Reduces hop count by adding another level of scale-out to Figure 6 which interconnects ring-stations.

An important insight from this study is that memory customization enables model- and deployment-specific system design. For instance, an RPU designed for the high-performance edge running Llama3-70B achieves 3.5ms/token using a memory configuration with BW/Cap=227 at 220W TDP. Separately, an edge-optimized system for Llama4-Maverick achieves 1.1ms/token at BW/Cap=38 (comparable to HBM3e) and 260W TDP. In contrast, for datacenter-scale deployments targeting 1kW TDP, we design an RPU for

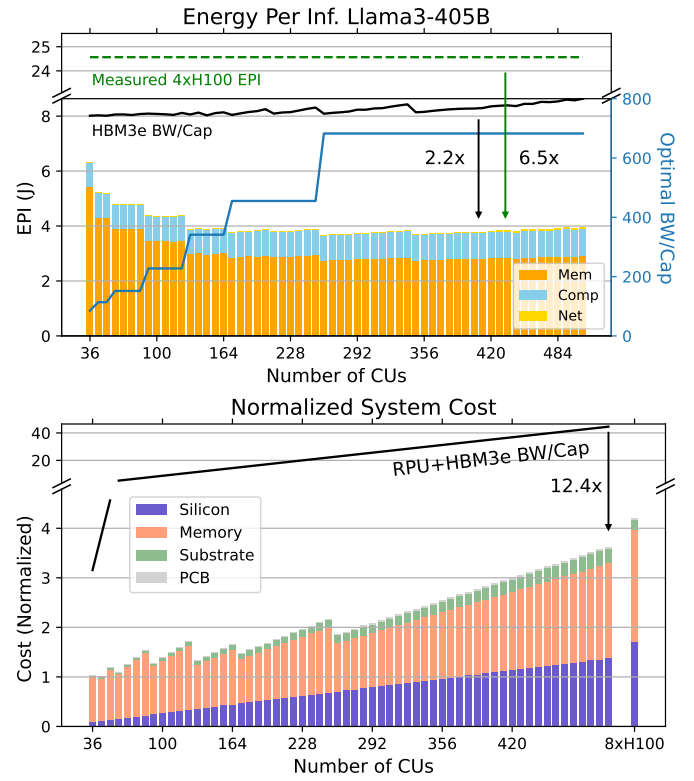


Fig. 12. Energy and cost analysis for scales of CUs running Llama3-405B at batch size 1. Top: Energy per inference and optimal memory selection. Bottom: Normalized system cost.

Llama3-70B with more CUs that achieves 0.65ms/token with a BW/Cap of 682 (the highest in our design space) while a datacenter RPU for Llama4-Maverick reaches 0.24ms/token at BW/Cap=170.

Batched Inference on the RPU: Figure 11 (bottom left) compares output tokens/sec per query across Llama models using a 128 CU RPU and an 8xH200 baseline reported by [4]. Llama4-Scout achieves the highest throughput across all batch sizes, closely followed by Llama4-Maverick. Activating more unique experts in Maverick reduces per-expert parallelism, leading to a 1.2-1.3x decrease in performance compared to Scout’s 16-expert configuration.

As batch size increases, per-query throughput decreases primarily due to serialized KV cache computations. Figure 11 (bottom right) demonstrates this behavior, indicating all models operate in a memory bandwidth-bound regime up to a batch size of 8. Beyond this point, the Llama3-405B model becomes compute-bound, as its attention mechanism features a high arithmetic intensity (16 queries per KV head), saturating available compute resources.

In contrast, the Llama4 models maintain high memory bandwidth utilization (above 80%) up to batch size 128. Their attention design, with only 5 queries per KV head, and MoE structure, are low per-token arithmetic intensity. Our layer smoothing technique balances arithmetic intensity across compute-bound layers (e.g., shared projections and MLP

layers) and memory-bound layers (e.g., attention and MoE layers). Between the two Llama4 variants, Scout becomes compute-bound earlier due to heavier per-expert loads (only 16 experts) versus Maverick’s 128-expert setup distributes tokens across more experts, preserving memory-bandwidth demands.

Energy and Cost Analysis: Figure 12 (top) extends our strong scaling results by analyzing energy per inference for Llama3-405B at batch size 1. The majority of the energy is consumed by memory accesses, making memory selection a dominant factor in system efficiency. To minimize energy, we explore HBM-CO design points along the Pareto frontier (Section VII), selecting the BW/Cap ratio that best matches system scale and capacity requirements.

At smaller scales, lower BW/Cap memory modules are required to meet overall capacity needs, resulting in higher energy per inference. As the system scale increases, each CU stores a smaller fraction of the model, allowing for a higher BW/Cap memory with lower capacity and improved energy efficiency. Energy per inference improves steadily with scale until 268 CUs where the highest BW/Cap memory module in the design space is selected.

Compared to an HBM3e BW/Cap memory, the HBM-CO memory improves energy efficiency by up to 2.2 $\times$. Similarly, compared to a 4 $\times$ H100 system running Llama3-405B, the HBM-CO optimized configuration achieves 6.5 $\times$ lower energy per inference. Combined with the latency-optimized design point at 428 CUs, this translates to a 412 $\times$ improvement in energy-delay product (EDP) relative to the 4 $\times$ H100 baseline. This highlights the power of co-designing memory and compute around bandwidth per capacity to unlock both latency and efficiency at scale.

Figure 12 (bottom) shows the normalized total system cost broken down by silicon, memory, substrate, and PCB. Costs are normalized to the smallest valid configuration for Llama3-405B. As expected, compute cost grows linearly with CU count, while memory cost increases sublinearly due to adaptive HBM-CO selection. At each scale, the memory configuration is selected from the HBM-CO Pareto frontier using the highest BW/cap memory which satisfies the required capacity. Discrete jumps visible in the memory cost curve correspond to transitions between HBM-CO tiers. While high BW/Cap memories are more expensive per GB, they eliminate capacity over-provision. As a result, total system cost is reduced.

Compared to using a fixed HBM3e memory, the HBM-CO system reduces total cost by up to 12.4 $\times$. At scale, its memory-to-compute cost ratio matches that of an 8 $\times$ H100 DGX [2], demonstrating that HBM-CO enable efficient bandwidth scaling while keeping costs reasonable.

Energy per Inference versus Batch Size: Figure 13 shows speedup and energy-per-inference of the RPU over an H100 across batch sizes for Llama3-8B and Llama3-70B. Larger batch sizes improve the GPU’s compute efficiency. However, concurrent queries inflate the KV\$ cache latency, introducing phase imbalance that the GPU cannot hide. For the RPU, small 4k sequences limit the benefit of decoupled pipelines because weight computation dominates, leaving less room to overlap

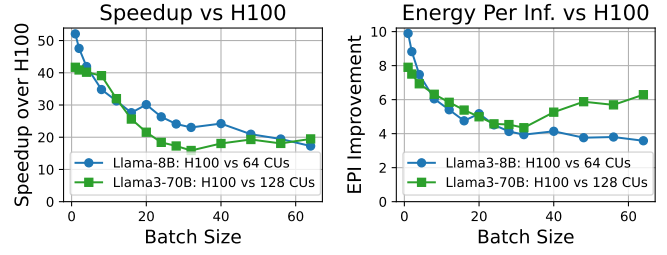


Fig. 13. Speedup and energy-per-inference of an RPU versus and H100, sweeping batch size for Llama models with 8k prefill 2k decode.

KV\$ prefetching. As a result, performance gains plateau at ~ 15 -20 $\times$ over the H100, though the RPU still maintains higher throughput and better energy efficiency.

At small batch sizes, the RPU shines, delivering over 40-50 $\times$ speedup and 8-10 $\times$ energy-per-inference, driven by its higher memory bandwidth and ability to efficiently execute small kernels with minimal synchronization overhead. In contrast, H100 performance performs poorly in this regime, as it is significantly bandwidth-bound and suffers from kernel launch and scheduling overheads.

IX. DECOMPOSED CONTRIBUTIONS

Contribution 1 – HBM-CO Memory: Compared to an RPU system using HBM3e, HBM-CO offers up to 2.2 $\times$ lower energy per inference and 12.4 $\times$ lower system cost, primarily by eliminating excess capacity and reducing internal wire lengths. These savings allow us to scale the number of compute units at ISO-TDP, leading to a 2.1 $\times$ latency improvement.

Contribution 2 – RPU Power and Area Provisioning: By rebalancing the compute-to-bandwidth ratio relative to an RPU provisioned like an H100 (~ 200 Ops/Byte), the RPU saves 3.3 $\times$ die cost and 2.6 $\times$ TDP utilization, leading to a 2.2 $\times$ latency improvement when scaling out at ISO-TDP.

Contribution 3 – Microarchitectural Decoupling: Fine-grained network sharding eliminates global synchronization, avoiding up to a 2.0 $\times$ latency penalty from collective stalls. Memory-compute decoupling enables deep prefetching, preventing a 1.2 $\times$ slowdown from serialized kernel execution. In batch size 32 workloads, decoupled execution allows the RPU to straddle the roofline across memory-bound (SDPA, MoE) and compute-bound (Linear) kernels, improving latency by up to 1.6 $\times$. These changes also improve energy efficiency: 1.4 $\times$ over a monolithic NUMA-style baseline via shorter data paths, and 1.7 $\times$ at the SRAM interface through on-the-fly stream dequantization. Together, these energy efficiency gains enable the system to scale to 2.4 $\times$ more bandwidth at ISO-TDP.

Cumulative Performance: HBM-CO, aligned provisioning, and decoupled pipelines enable 20-40 $\times$ higher effective memory bandwidth at ISO-TDP, consistent with simulation results.

RPU Application Domain: Human-computer interaction literature identifies an interaction-latency threshold on the order of ten seconds, beyond which working memory decays and users are likely to context-switch, incurring re-orientation overheads [21], [42], [44]. Accordingly, reasoning systems

must minimize end-to-end latency to preserve turn-taking and cognitive continuity, rather than maximizing throughput. This captures the motivation behind the RPU: we want advanced intelligence at our fingertips. The RPU targets these reasoning-intensive, interactive workloads requiring end-to-end responses such as multi-step planning, problem solving, iterative coding, and writing assistance, which currently take tens of seconds to minutes on today’s systems [56]. By exploiting the latency benefits of low-batch inference, it delivers state-of-the-art responsiveness and per-query performance.

X. RELATED WORK

DRAM-Centric General-Purpose Accelerators: Systems such as NVIDIA H100 [14], AMD MI300x [61], SambaNova [52], and TPU [26] use high-capacity HBM, large shared caches, and dense compute to support both training and inference. These architectures typically feature a single NUMA domain with distributed controllers and centralized caches. While this enables flexible data access, it creates long memory paths and high energy per access, which is especially harmful for memory-bound decode. In contrast, the RPU uses a fine-grained NUMA design where each core has its own HBM-CO DRAM channel and local SRAM buffer, eliminating shared caches and reducing on-chip data movement. This decouples compute, memory, and network pipelines, sustaining high bandwidth utilization.

SRAM-Centric Custom Accelerators: Custom accelerators such as Groq [2], Cerebras WSE-3 [64], and Graphcore IPU [32] rely on SRAM as main memory. However, the limited density of SRAM makes it impractical to store large models efficiently. For example, a 70B parameter model deployed on Groq requires hundreds of accelerator cards, while Cerebras spans four wafer-scale chips.

To utilize their full SRAM bandwidth, these systems shard each matrix across a large compute fabric. For example, Cerebras may distribute a single VMM across 900,000 cores, requiring vector broadcasts to traverse up to 1,000 core-to-core hops and reductions to span the entire wafer. With the model globally distributed, network communication, not compute or memory access, becomes the primary performance bottleneck due to SRAM’s low density. In contrast, each RPU reasoning core is significantly more capable than the ultra-lightweight cores used in Groq or Cerebras, with higher FLOP throughput and wider data buses. As a result, more of the workload is processed locally per core, reducing reliance on multi-hop communication. Additionally, the RPU’s hierarchical ring network has a much smaller diameter than mesh or wafer-scale fabrics, further minimizing the number of hops required for vector broadcasts and reductions.

Processing In Memory (PIM) Accelerators: PIM architectures aim to reduce data movement by embedding compute capabilities within or near memory [13], [24], [36], [46], [70]. Many PIM designs leverage DRAM or emerging memory technologies to perform simple operations, typically integer or bitwise logic, in situ. While effective for low-intensity workloads, PIM designs struggle when arithmetic intensity

System	System Metrics			Speculative Decoding			
	Main Memory	BW/Cap (1/s)	Comp/BW (Ops/Byte)	Systems (Spec-70B)	TDP (W)	Shoreline (mm)	Perf (Tokens/s)
NVIDIA H200	HBM3e	34 ↓	206 ↑	1 -	700 ↓	66 ↓	134 ↓
SambaNova	HBM3	25 ↓	399 ↑	16 -	10k ↑	704 ↓	457 ↓
Groq LPU	SRAM	355k ↑	2.4 ↓	500 ‡	100k ↑	NA	1678 ↓
Cerebras WSE-3	SRAM	477k ↑	6.0 ↓	4 -	136k ↑	NA	2148 ↓
RPU	HBM-CO	500 ↑	32 ↑	200CU -	1.8k ↑	1500 ↑	4423 ↑

‡ Groq est. 400-600 Processors.

Fig. 14. A comparison of leading hardware platforms. Speculative decoding throughput for Llama3-70B based on published data [2], [52], [57], [64].

exceeds 1 Op/Byte, which is common during LLM inference. PIM architectures are also poorly suited for floating-point operations or fine-grained programmability to support rotary embeddings, softmax, and normalization functions.

Furthermore, the rise of block-quantized formats (e.g., BFP, MXFP) poses a major challenge for PIM. These formats require dynamic exponent broadcasting, alignment, and decoding before arithmetic. These steps involve conditional logic and variable indexing, which are difficult to implement in DRAM-compatible circuitry.

Comparison Under Speculative Decoding: Speculative decoding is an increasingly common technique used in LLM inference to reduce token generation latency by leveraging a lightweight “draft” model to predict multiple tokens ahead. These predicted tokens are then validated by a larger “target” model; if the predictions are correct, several tokens can be committed in parallel. This approach may be challenging because it increases the arithmetic intensity of each query.

Industry accelerators often report performance under speculative decoding. We evaluate the RPU using a comparable speculative decode setup. In our evaluation, we adopt an 8-token lookahead configuration in which a Llama3-8B draft model proposes tokens for a Llama3-70B target model. On average, 4.6 tokens are accepted per speculative window [41], accelerating end-to-end inference by 1.8×. Figure 14 compares our speculative performance to publicly reported numbers from NVIDIA H200 [57], SambaNova SN40L [52], Groq [2], and Cerebras WSE-3 [64]. The RPU-200U configuration is lower latency than all evaluated systems.

XI. CONCLUSION

Just as custom logic design led to configurable ASIC toolchains and foundry ecosystems, memory must follow a similar path. Commodity DRAMs no longer aligns with the needs of modern inference systems. Memory should be treated not as fixed infrastructure, but as a key dimension of system specialization. With chiplet-based integration, customizable memory is not only desirable but also practical. It allows system designers to co-optimize bandwidth, capacity, and energy by tailoring the design, packaging, and tuning of DRAM components for specific workloads. The RPU system embraces this philosophy by co-designing its memory architecture with compute and interconnect, treating memory not as a constraint but as an opportunity for specialization.

REFERENCES

- [1] "Introduction to torch.compile — PyTorch Tutorials 2.6.0+cu124 documentation," 2023. [Online]. Available: https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html
- [2] "Groq First Generation 14nm Chip Just Got a 6x Speed Boost: Introducing Llama 3.3 70B Speculative Decoding on GroqCloud™ - Groq is Fast AI Inference," Nov. 2024, section: Blog. [Online]. Available: <https://groq.com/groq-first-generation-14nm-chip-just-got-a-6x-speed-boost-introducing-llama-3-1-70b-speculative-decoding-on-groqcloud/>
- [3] "NVIDIA Data Center Deep Learning Product Performance AI Inference," 2024. [Online]. Available: <https://developer.nvidia.com/deep-learning-performance-training-inference/ai-inference>
- [4] "Hardware Benchmarking & Performance Analysis," Jun. 2025. [Online]. Available: <https://artificialanalysis.ai/benchmarks/hardware>, <https://artificialanalysis.ai/benchmarks/hardware>
- [5] "neuralmagic (Neural Magic)," Feb. 2025. [Online]. Available: <https://huggingface.co/neuralmagic>
- [6] "NVIDIA Blackwell," 2025. [Online]. Available: <https://nvdam.widen.net/s/wnsrxrm2w/blackwell-datasheet-3384703>
- [7] A. Akram and L. Sawalha, "A Survey of Computer Architecture Simulation Techniques and Tools," *IEEE Access*, vol. 7, pp. 78 120–78 145, 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8718630>
- [8] M. Ballon, A. Algaba, and V. Ginis, "The Relationship Between Reasoning and Performance in Large Language Models – o3 (mini) Thinks Harder, Not Longer," Feb. 2025, arXiv:2502.15631 [cs] version: 1. [Online]. Available: <http://arxiv.org/abs/2502.15631>
- [9] S. Campanoni, K. Brownell, S. Kanev, T. M. Jones, G.-Y. Wei, and D. Brooks, "HELIX-RC: an architecture-compiler co-design for automatic parallelization of irregular programs," *SIGARCH Comput. Archit. News*, vol. 42, no. 3, pp. 217–228, Jun. 2014. [Online]. Available: <https://doi.org/10.1145/2678373.2665705>
- [10] B. Canakci, J. Liu, X. Wu, N. Cheriere, P. Costa, S. Legtchenko, D. Narayanan, and A. Rowstron, "Good things come in small packages: Should we build AI clusters with Lite-GPUs?" Apr. 2025, arXiv:2501.10187 [cs]. [Online]. Available: <http://arxiv.org/abs/2501.10187>
- [11] N. Chatterjee, M. O'Connor, D. Lee, D. R. Johnson, S. W. Keckler, M. Rhu, and W. J. Dally, "Architecting an Energy-Efficient DRAM System for GPUs," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb. 2017, pp. 73–84, iSSN: 2378-203X. [Online]. Available: <https://ieeexplore.ieee.org/document/7920815/?arnumber=7920815>
- [12] X. Chen, J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang, R. Wang, Z. Tu, H. Mi, and D. Yu, "Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs," Feb. 2025, arXiv:2412.21187 [cs]. [Online]. Available: <http://arxiv.org/abs/2412.21187>
- [13] J. Choi, J. Park, K. Kyung, N. S. Kim, and J. H. Ahn, "Unleashing the Potential of PIM: Accelerating Large Batched Inference of Transformer-Based Generative Models," *IEEE Computer Architecture Letters*, vol. 22, no. 2, pp. 113–116, Jul. 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10218731>
- [14] J. Choquette, "NVIDIA Hopper H100 GPU: Scaling Performance," *IEEE Micro*, vol. 43, no. 3, pp. 9–17, May 2023, conference Name: IEEE Micro.
- [15] B. Darvish Rouhani, R. Zhao, V. Elango, R. Shafipour, M. Hall, M. Mesmakhosroshahi, A. More, L. Melnick, M. Golub, G. Varatkar, L. Shao, G. Kolhe, D. Melts, J. Klar, R. L'Heureux, M. Perry, D. Burger, E. Chung, Z. S. Deng, S. Naghshtineh, J. Park, and M. Naumov, "With Shared Microexponents, A Little Shifting Goes a Long Way," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, Jun. 2023, pp. 1–13.
- [16] D. Das Sharma, G. Pasdast, Z. Qian, and K. Aygun, "Universal Chiplet Interconnect Express (UCIe): An Open Industry Standard for Innovations With Chiplets at Package Level," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 12, no. 9, pp. 1423–1431, Sep. 2022, conference Name: IEEE Transactions on Components, Packaging and Manufacturing Technology. [Online]. Available: <https://ieeexplore.ieee.org/document/9893865>
- [17] A. Elmeleegy, H. Kim, D. Zier, K. Kranen, N. Shah, R. Olson, and O. Kahalon, "Introducing NVIDIA Dynamo, A Low-Latency Distributed Inference Framework for Scaling Reasoning AI Models," Mar. 2025. [Online]. Available: <https://developer.nvidia.com/blog/introducing-nvidia-dynamo-a-low-latency-distributed-inference-framework-for-scaling-reasoning-ai-models/>
- [18] E. Frantar, R. L. Castro, J. Chen, T. Hoefer, and D. Alistarh, "MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models," in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP '25. New York, NY, USA: Association for Computing Machinery, Feb. 2025, pp. 239–251. [Online]. Available: <https://dl.acm.org/doi/10.1145/3710848.3710871>
- [19] A. Frumusanu, "TSMC Details 3nm Process Technology: Full Node Scaling for 2H22 Volume Production," Aug. 2020. [Online]. Available: <https://www.anandtech.com/show/16024/tsmc-details-3nm-process-technology-details-full-node-scaling-for-2h22>
- [20] S. Gandhi, P. v. Platen, and A. M. Rush, "Distil-Whisper: Robust Knowledge Distillation via Large-Scale Pseudo Labelling," Nov. 2023, arXiv:2311.00430 [cs]. [Online]. Available: <http://arxiv.org/abs/2311.00430>
- [21] U. Gnewuch, S. Morana, M. T. P. Adam, and A. Maedche, "Opposing Effects of Response Time in Human–Chatbot Interaction," *Business & Information Systems Engineering*, vol. 64, no. 6, pp. 773–791, Dec. 2022. [Online]. Available: <https://doi.org/10.1007/s12599-022-00755-x>
- [22] W. Gomes, A. Koker, P. Stover, D. Ingerly, S. Siers, S. Venkataraman, C. Peltó, T. Shah, A. Rao, F. O'Mahony, E. Karl, L. Cheney, I. Rajwani, H. Jain, R. Cortez, A. Chandrasekhar, B. Kanthi, and R. Koduri, "Ponte Vecchio: A Multi-Tile 3D Stacked Processor for Exascale Computing," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, Feb. 2022, pp. 42–44, iSSN: 2376-8606. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9731673>
- [23] P. Gratz, C. Kim, K. Sankaralingam, H. Hanson, P. Shivakumar, S. W. Keckler, and D. Burger, "On-Chip Interconnection Networks of the TRIPS Chip," *IEEE Micro*, vol. 27, no. 5, pp. 41–50, Sep. 2007, conference Name: IEEE Micro. [Online]. Available: <https://ieeexplore.ieee.org/document/4378782/?arnumber=4378782>
- [24] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, Apr. 2024, pp. 722–737, arXiv:2403.00579 [cs]. [Online]. Available: <http://arxiv.org/abs/2403.00579>
- [25] Y.-C. Hu, Y.-M. Liang, H.-P. Hu, C.-Y. Tan, C.-T. Shen, C.-H. Lee, and S. Y. Hou, "CoWoS Architecture Evolution for Next Generation HPC on 2.5D System in Package," in *2023 IEEE 73rd Electronic Components and Technology Conference (ECTC)*. Orlando, FL, USA: IEEE, May 2023, pp. 1022–1026. [Online]. Available: <https://ieeexplore.ieee.org/document/10195565/>
- [26] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson, "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, Jun. 2023, pp. 1–14. [Online]. Available: <https://dl.acm.org/doi/10.1145/3579371.3589350>
- [27] R. Kaplan, "Intel Gaudi 3 AI Accelerator: Architected for Gen AI Training and Inference," in *2024 IEEE Hot Chips 36 Symposium (HCS)*. Stanford, CA, USA: IEEE, Aug. 2024, pp. 1–16. [Online]. Available: <https://ieeexplore.ieee.org/document/10665178/>
- [28] J.-Y. Kim, T. Kim, J. You, K. Kim, B. M. Moon, K. Sohn, and S.-O. Jung, "An Energy-Efficient Design of TSV I/O for HBM With a Data Rate up to 10 Gb/s," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 11, pp. 3242–3252, Nov. 2023, conference Name: IEEE Journal of Solid-State Circuits. [Online]. Available: <https://ieeexplore.ieee.org/document/10164008>
- [29] K. Kim and M.-j. Park, "Present and Future, Challenges of High Bandwidth Memory (HBM)," in *2024 IEEE International Memory Workshop (IMW)*, May 2024, pp. 1–4, iSSN: 2573-7503. [Online]. Available: <https://ieeexplore.ieee.org/document/10536972>
- [30] Y. Kim, V. Seshadri, D. Lee, J. Liu, and O. Mutlu, "A case for exploiting subarray-level parallelism (SALP) in DRAM," in *2012 39th*

- Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2012, pp. 368–379, iSSN: 1063-6897. [Online]. Available: <https://ieeexplore.ieee.org/document/6237032>
- [31] Y. Kim, V. Seshadri, D. Lee, J. Liu, and O. Mutlu, “Exploiting the DRAM Microarchitecture to Increase Memory-Level Parallelism,” May 2018, arXiv:1805.01966 [cs]. [Online]. Available: <http://arxiv.org/abs/1805.01966>
- [32] S. Knowles, “Graphcore,” in *2021 IEEE Hot Chips 33 Symposium (HCS)*, Aug. 2021, pp. 1–25, iSSN: 2573-2048. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9567075>
- [33] J. Kundu, W. Guo, A. BanaGoza, U. De Alwis, S. Sengupta, P. Gupta, and A. Mallik, “Performance Modeling and Workload Analysis of Distributed Large Language Model Training and Inference,” in *2024 IEEE International Symposium on Workload Characterization (IISWC)*, Sep. 2024, pp. 57–67, iSSN: 2835-2238. [Online]. Available: <https://ieeexplore.ieee.org/document/10763669/?arnumber=10763669>
- [34] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” Sep. 2023, arXiv:2309.06180 [cs]. [Online]. Available: <http://arxiv.org/abs/2309.06180>
- [35] J. Lee, K. Cho, C. K. Lee, Y. Lee, J.-H. Park, S.-H. Oh, Y. Ju, C. Jeong, H. S. Cho, J. Lee, T.-S. Yun, J. H. Cho, S. Oh, J. Moon, Y.-J. Park, H.-S. Choi, I.-K. Kim, S. M. Yang, S.-Y. Kim, J. Jang, J. Kim, S.-H. Lee, Y. Jeon, J. Park, T.-K. Kim, D. Ka, S. Oh, J. Kim, J. Jeon, S. Kim, K. T. Kim, H. Yang, D. Yang, M. Lee, H. Song, D. Jang, J. Shin, H. Kim, C. Baek, H. Jeong, J. Yoon, S.-K. Lim, K. Y. Lee, Y. J. Koo, M.-J. Park, J. Cho, and J. Kim, “13.4 A 48GB 16-High 1280GB/s HBM3E DRAM with All-Around Power TSV and a 6-Phase RDQS Scheme for TSV Area Optimization,” in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, Feb. 2024, pp. 238–240, iSSN: 2376-8606. [Online]. Available: <https://ieeexplore.ieee.org/document/10454440>
- [36] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2021, pp. 43–56, iSSN: 2575-713X. [Online]. Available: <https://ieeexplore.ieee.org/document/9499894/>
- [37] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” May 2023, arXiv:2211.17192 [cs]. [Online]. Available: <http://arxiv.org/abs/2211.17192>
- [38] Z. Liu and V. Grover, “A Performance Model for Warp Specialization Kernels,” Jun. 2025, arXiv:2506.11209 [cs]. [Online]. Available: <http://arxiv.org/abs/2506.11209>
- [39] Y.-C. Lo, G.-Y. Wei, and D. Brooks, “Nanoscaling Floating-Point (NxFP): NanoMantissa, Adaptive Microexponents, and Code Recycling for Direct-Cast Compression of Large Language Models,” Dec. 2024, arXiv:2412.19821 [cs].
- [40] R. Mahajan, R. Sankman, N. Patel, D.-W. Kim, K. Aygun, Z. Qian, Y. Mekonnen, I. Salama, S. Sharan, D. Iyengar, and D. Mallik, “Embedded Multi-die Interconnect Bridge (EMIB) – A High Density, High Bandwidth Packaging Interconnect,” in *2016 IEEE 66th Electronic Components and Technology Conference (ECTC)*, May 2016, pp. 557–565. [Online]. Available: <https://ieeexplore.ieee.org/document/7545486>
- [41] J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat, and R. Schwartz, “Accelerating Speculative Decoding using Dynamic Speculation Length,” May 2024, arXiv:2405.04304 [cs] version: 1. [Online]. Available: <http://arxiv.org/abs/2405.04304>
- [42] M. Maslych, M. Katebi, C. Lee, Y. Hmaiti, A. Ghasemaghahi, C. Pumarada, J. Palmer, E. S. Martinez, M. Emporio, W. Snipes, R. P. McMahan, and J. J. L. Jr, “Mitigating Response Delays in Free-Form Conversations with LLM-powered Intelligent Virtual Agents,” in *Proceedings of the 7th ACM Conference on Conversational User Interfaces*, Jul. 2025, pp. 1–15, arXiv:2507.22352 [cs]. [Online]. Available: <http://arxiv.org/abs/2507.22352>
- [43] K.-I. Moon, H.-Y. Son, and K. Lee, “Advanced Packaging Technologies in Memory Applications for Future Generative AI Era,” in *2023 International Electron Devices Meeting (IEDM)*, Dec. 2023, pp. 1–4, iSSN: 2156-017X. [Online]. Available: <https://ieeexplore.ieee.org/document/10413890>
- [44] J. Nielsen, “Slow AI: Designing User Control for Long Tasks,” Oct. 2025. [Online]. Available: <https://jakobnielsenphd.substack.com/p/slow-ai>
- [45] M. O’Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, “Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems,” in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 2017, pp. 41–54, iSSN: 2379-3155. [Online]. Available: <https://ieeexplore.ieee.org/document/8686544>
- [46] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24, vol. 2. New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 103–119. [Online]. Available: <https://doi.org/10.1145/3620665.3640422>
- [47] M.-J. Park, J. Lee, K. Cho, J. Park, J. Moon, S.-H. Lee, T.-K. Kim, S. Oh, S. Choi, Y. Choi, H. S. Cho, T. Yun, Y. J. Koo, J.-S. Lee, B.-K. Yoon, Y.-J. Park, S. Oh, C. K. Lee, S.-H. Lee, H.-W. Kim, Y. Ju, S.-K. Lim, K. Y. Lee, S.-H. Lee, W. S. We, S. Kim, S. M. Yang, K. Lee, I.-K. Kim, Y. Jeon, J.-H. Park, J. C. Yun, S. Kim, D.-Y. Lee, S.-H. Oh, J.-H. Shin, Y. Lee, J. Jang, and J. Cho, “A 192-Gb 12-High 896-GB/s HBM3 DRAM With a TSV Auto-Calibration Scheme and Machine-Learning-Based Layout Optimization,” *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 256–269, Jan. 2023, conference Name: IEEE Journal of Solid-State Circuits. [Online]. Available: <https://ieeexplore.ieee.org/document/9858112/?arnumber=9858112>
- [48] D. Patel, J. Koch, T. Bennett, W. Chu, and A. Ahmad, “The Memory Wall: Past, Present, and Future of DRAM,” Sep. 2024. [Online]. Available: <https://semianalysis.com/2024/09/03/the-memory-wall/>
- [49] D. Patel and G. Wong, “AI Server Cost Analysis – Memory Is The Biggest Loser,” May 2023. [Online]. Available: <https://semianalysis.com/2023/05/29/ai-server-cost-analysis-memory-is/>
- [50] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” May 2024, arXiv:2311.18677 [cs].
- [51] J. W. Poulton, J. M. Wilson, W. J. Turner, B. Zimmer, X. Chen, S. S. Kudva, S. Song, S. G. Tell, N. Nedovic, W. Zhao, S. R. Sudhakaran, C. T. Gray, and W. J. Dally, “A 1.17-pJ/b, 25-Gb/s/pin Ground-Referenced Single-Ended Serial Link for Off- and On-Package Communication Using a Process- and Temperature-Adaptive Voltage Regulator,” *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 43–54, Jan. 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8528390/>
- [52] R. Prabhakar, R. Sivaramakrishnan, D. Gandhi, Y. Du, M. Wang, X. Song, K. Zhang, T. Gao, A. Wang, X. Li, Y. Sheng, J. Brot, D. Sokolov, A. Vivek, C. Leung, A. Sabnis, J. Bai, T. Zhao, M. Gottscho, D. Jackson, M. Luttrell, M. K. Shah, Z. Chen, K. Liang, S. Jain, U. Thakker, D. Huang, S. Jairath, K. J. Brown, and K. Olukotun, “SambaNova SN40L: Scaling the AI Memory Wall with Dataflow and Composition of Experts,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Nov. 2024, pp. 1353–1366, iSSN: 2379-3155. [Online]. Available: <https://ieeexplore.ieee.org/document/10764648>
- [53] B. Rouhani, D. Lo, R. Zhao, M. Liu, J. Fowers, K. Ovtcharov, A. Vinogradsky, S. Massengill, L. Yang, R. Bittner, A. Forin, H. Zhu, T. Na, P. Patel, S. Che, L. C. Koppaka, X. Song, S. Som, K. Das, S. Tiwary, S. Reinhardt, S. Lanka, E. Chung, and D. Burger, “Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point,” ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., Dec. 2020, pp. 10271–10281.
- [54] Y. Ryu, S.-G. Ahn, J. H. Lee, J. Park, Y. K. Kim, H. Kim, Y. G. Song, H.-W. Cho, S. Cho, S. H. Song, H. Lee, U. Shin, J. Ahn, J.-M. Ryu, S. Lee, K.-H. Lim, J. Lee, J. H. Park, J.-S. Jeong, S. Joo, D. Cho, S. Y. Kim, M. Lee, H. Kim, M. Kim, J.-S. Kim, J. Kim, H. G. Kang, M.-K. Lee, S.-R. Kim, Y.-C. Kwon, Y. Y. Byun, K. Lee, S. Park, J. Youn, M.-O. Kim, K. Sohn, S.-J. Hwang, and J. Lee, “A 16 GB 1024 GB/s HBM3 DRAM With Source-Synchronized Bus Design and On-Die Error Control Scheme for Enhanced RAS Features,” *IEEE Journal of Solid-State Circuits*, vol. 58, no. 4, pp. 1051–1061, Apr. 2023, conference Name: IEEE Journal of Solid-State Circuits. [Online]. Available: <https://ieeexplore.ieee.org/document/10005600>
- [55] D. Sanchez and C. Kozyrakis, “ZSim: fast and accurate microarchitectural simulation of thousand-core systems,” *SIGARCH*

- Comput. Archit. News*, vol. 41, no. 3, pp. 475–486, Jun. 2013. [Online]. Available: <https://doi.org/10.1145/2508148.2485963>
- [56] SemiAnalysis, “InferenceMAX by SemiAnalysis.” [Online]. Available: <https://inferencemax.semianalysis.com>
- [57] A. Shah, A. Eassa, C. Putterman, L. Vaidya, J. Gangani, and A. Srivastava, “Boost Llama 3.3 70B Inference Throughput 3x with NVIDIA TensorRT-LLM Speculative Decoding,” Dec. 2024. [Online]. Available: <https://developer.nvidia.com/blog/boost-llama-3-3-70b-inference-throughput-3x-with-nvidia-tensorrt-llm-speculative-decoding/>
- [58] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. Emer, C. T. Gray, B. Khailany, and S. W. Keckler, “Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’52. New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 14–27.
- [59] A. Shilov, “TSMC Details 5 nm Process Tech: Aggressive Scaling, But Thin Power and Performance Gains,” May 2018. [Online]. Available: <https://www.anandtech.com/show/12727/tsmc-details-5-nm-process-tech-aggressive-scaling-but-thin-power-and-performance-gains>
- [60] A. Shilov, “TSMC’s Roadmap at a Glance: N3X, N2P, A16 Coming in 2025/2026,” May 2024. [Online]. Available: <https://www.anandtech.com/show/21408/tsmc-roadmap-at-a-glance-n3x-n2p-a16-2025-2026>
- [61] A. Smith, G. H. Loh, J. Wu, S. Naffziger, T. Huang, H. McIntyre, R. Mangaser, W. Jung, and R. Swaminathan, “AMD Instinct™ MI300X Accelerator: Packaging and Architecture Co-Optimization,” in *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, Jun. 2024, pp. 1–2, iSSN: 2158-9682. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10631545>
- [62] T. Tambe, C. Hooper, L. Pentecost, T. Jia, E.-Y. Yang, M. Donato, V. Sanh, P. Whatmough, A. M. Rush, D. Brooks, and G.-Y. Wei, “EdgeBERT: Sentence-Level Energy Optimizations for Latency-Aware Multi-Task NLP Inference,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’21. New York, NY, USA: Association for Computing Machinery, Oct. 2021, pp. 830–844. [Online]. Available: <https://dl.acm.org/doi/10.1145/3466752.3480095>
- [63] A. Tirumala and R. Wong, “NVIDIA Blackwell Platform: Advancing Generative AI and Accelerated Computing,” in *2024 IEEE Hot Chips 36 Symposium (HCS)*, Aug. 2024, pp. 1–33, iSSN: 2573-2048. [Online]. Available: <https://ieeexplore.ieee.org/document/10665247>
- [64] J. Wang, “Cerebras Inference now 3x faster: Llama3.1-70B breaks 2,100 tokens/s - Cerebras,” Oct. 2024. [Online]. Available: <https://www.cerebras.ai/blog/cerebras-inference-3x-faster>
- [65] S. Wang, J. Wei, A. Sabne, A. Davis, B. Ilbeyi, B. Hechtman, D. Chen, K. S. Murthy, M. Maggioni, Q. Zhang, S. Kumar, T. Guo, Y. Xu, and Z. Zhou, “Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models.” Vancouver BC Canada: ACM, Dec. 2022, pp. 93–106.
- [66] Y. Wang, Q. Liu, J. Xu, T. Liang, X. Chen, Z. He, L. Song, D. Yu, J. Li, Z. Zhang, R. Wang, Z. Tu, H. Mi, and D. Yu, “Thoughts Are All Over the Place: On the Underthinking of o1-Like LLMs,” Feb. 2025, arXiv:2501.18585 [cs]. [Online]. Available: <http://arxiv.org/abs/2501.18585>
- [67] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” Jan. 2023, arXiv:2201.11903 [cs].
- [68] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, “LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2024, pp. 1080–1096. [Online]. Available: <https://ieeexplore.ieee.org/document/10609604>
- [69] S. Q. Zhang, T. Tambe, N. Cuevas, G.-Y. Wei, and D. Brooks, “CAMEL: Co-Designing AI Models and eDRAMs for Efficient On-Device Learning,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Mar. 2024, pp. 861–875, iSSN: 2378-203X. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10476409>
- [70] M. Zhou, W. Xu, J. Kang, and T. Rosing, “TransPIM: A Memory-based Acceleration via Software-Hardware Co-Design for Transformer,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Apr. 2022, pp. 1071–1085, iSSN: 2378-203X. [Online]. Available: <https://ieeexplore.ieee.org/document/9773212>
- [71] B. Zimmer, R. Venkatesan, Y. S. Shao, J. Clemons, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. S. Emer, C. T. Gray, S. W. Keckler, and B. Khailany, “A 0.11 pJ/Op, 0.32-128 TOPS, Scalable Multi-Chip-Module-based Deep Neural Network Accelerator with Ground-Reference Signaling in 16nm,” in *2019 Symposium on VLSI Circuits*, Jun. 2019, pp. C300–C301, iSSN: 2158-5636. [Online]. Available: <https://ieeexplore.ieee.org/document/8778056>